

The 10th International Conference on Computer Science and Computational Intelligence 2025

A recruitment support system based on Large Language Models and Retrieval-Augmented Generation

An Cong Tran^{a,*}, Nghi Cong Tran^a

^aCan Tho University, Can Tho 90000, Vietnam

Abstract

This paper proposes a method for building a recruitment support system that leverages Retrieval-Augmented Generation (RAG) and large language models to automate the processing and analysis of job applications. The system employs a modular architecture that integrates a vector database for semantic retrieval, agent-based query processing, and cloud-based storage and computation to ensure scalability and cost-efficiency. The implementation techniques include extracting data from PDF applications, chunking text into 256-token segments, embedding with Amazon Titan Text Embeddings v2, and generating responses using Anthropic Claude 3 Haiku with a 200,000-token context window. Deployed on AWS, the system optimizes resource usage by employing serverless computing. We evaluated the system using a dataset of 400 test cases, curated from 800 curriculum vitae, utilizing the RAGAS framework. The results demonstrate strong performance, achieving a Context Recall of 0.9183, Faithfulness of 0.882, Factual Correctness of 0.753, and Semantic Similarity of 0.827. A user-friendly interface supports semantic searches, keyword queries, and direct CV interactions, which enhances practical applicability. The system effectively streamlines candidate screening, with high retrieval accuracy and response fidelity.

© 2025 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 10th International Conference on Computer Science and Computational Intelligence (ICCSCI) 2025

Keywords: recruitment automation; support system; retrieval-augmented generation; large language models; semantic search; AWS deployment.

1. Introduction

Rapid advancements in information technology have made automating recruitment processes essential for organizations aiming to streamline talent acquisition. However, analyzing and evaluating a large volume of job

* Corresponding author.

E-mail address: tcn@ctu.edu.vn

applications, which often contain heterogeneous and unstructured data, remains a significant challenge. Traditional recruitment methods, predominantly reliant on manual screening, are increasingly strained by the growing number of applicants and the nuanced, often subjective nature of job requirements. These challenges underscore the urgent need for intelligent, scalable systems capable of processing unstructured data and delivering precise candidate matches, thereby addressing inefficiencies and biases inherent in conventional approaches. The application of machine learning (ML) and natural language processing (NLP) to recruitment systems has thus attracted significant interest with prior studies focusing on automating various stages of the process, including resume classification, candidate competency evaluation, and candidate-job matching.

Early efforts in this domain leveraged traditional ML techniques to automate recruitment tasks. For instance, Javed et al. developed Carotene, a job title classification system for online recruitment, using Support Vector Machines (SVM) and logistic regression [1]. Their system achieved an accuracy of 85% in classifying job titles into predefined categories on a dataset of 15,000 job postings, demonstrating the potential of ML for structured data processing. However, its reliance on manual feature engineering limited its ability to handle unstructured text, such as detailed resume content, and it struggled to adapt to diverse job roles. The advent of advanced NLP models shifted the focus toward more sophisticated approaches. Devlin et al. introduced BERT (Bidirectional Encoder Representations from Transformers), which achieved an F1-score of 88% in candidate-job matching on a dataset of 20,000 job applications in follow-up studies [2]. BERT's strength lies in its ability to capture bidirectional context, but its computational intensity and struggles with noisy data pose challenges for practical deployment. Similarly, Brown et al. developed GPT-3 and GPT-4, which demonstrated a 91% accuracy in generating contextually appropriate responses for tasks like candidate feedback on a test set of 10,000 interactions [3]. While versatile, these models lack retrieval mechanisms to round responses, limiting their effectiveness for precise matching. Recent works have further improved recruitment automation using machine learning and NLP approaches [4, 5, 6].

To address these limitations, Retrieval-Augmented Generation (RAG), proposed by Lewis et al., combines information retrieval with text generation, achieving a precision of 90% and a recall of 87% in extracting relevant candidate profiles from a dataset of 50,000 resumes [7]. RAG's ability to retrieve relevant documents before generating responses enhances the relevance of recommendations, though its performance depends heavily on the quality of the retrieval step. Semantic search has also been pivotal, with Mikolov et al. introducing word2vec, which improved candidate matching by 15% compared to keyword-based methods by achieving a cosine similarity score of 0.75 on benchmark datasets [8]. Subsequent models like GloVe [9] and Sentence-BERT [10] further advanced semantic search, with Sentence-BERT reporting an F1-score of 92% on a dataset of 30,000 job postings. However, these models require significant computational resources, posing challenges for resource-constrained environments. Qin et al. integrated NLP with semantic techniques, achieving a precision of 90% and a recall of 86% in matching candidates to job requirements on a dataset of 120,000 applications, but their system's scalability was limited due to its reliance on a single neural network architecture [11]. RAG approaches have shown effectiveness in many domains, including QA and medical applications [12, 13].

Supporting technologies have also played a crucial role. Chaudhuri provided foundational insights into query optimization in relational databases, reducing query execution time by up to 40% on benchmark datasets, which is critical for systems using PostgreSQL with extensions like pgvector for semantic search [14]. Cloud computing, as explored by Buyya et al., enables scalable deployment of ML applications, reducing operational costs by approximately 30%, though latency challenges remain for real-time applications [15]. Additionally, Xiong et al. demonstrated the utility of Optical Character Recognition (OCR) for document processing, achieving a 93.2% accuracy in text extraction on 3,000 documents, though performance dropped with low-quality scans [16]. While these studies provide a strong foundation, they often address specific aspects of recruitment in isolation, leaving gaps in scalability, semantic understanding, and bias mitigation.

This study proposes a recruitment support system that integrates Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to optimize candidate search and evaluation based on specific job requirements. LLMs, such as Claude Haiku and Claude Sonnet, hosted on Amazon Bedrock, enable the processing of unstructured job applications by extracting key candidate attributes and transforming them into semantic representations for accurate matching. RAG enhances this process by retrieving relevant applications from a database and generating semantically coherent responses, while semantic search, supported by vector embeddings, ensures matches are based on meaning rather than keywords. The system is deployed on Amazon Web Services (AWS), utilizing AWS Amplify for the

frontend interface, Amazon Cognito for secure user authentication, Amazon API Gateway and AWS Lambda for request processing and serverless RAG functions, and Amazon S3 for storing candidate data in PDF or image formats. Amazon Textract is employed as an OCR tool to extract text from unstructured documents, which are then managed in Amazon PostgreSQL with the pgvector extension for efficient semantic search. Titan embedding models from Amazon Bedrock convert applications and job requirements into semantic vectors, enabling precise candidate evaluation based on semantic similarity.

This research presents several important contributions: (1) it introduces a methodology that combines large language models (LLMs), retrieval-augmented generation (RAG), semantic search, and optical character recognition (OCR) to improve the accuracy and explainability of candidate matching; (2) it implements this methodology on a scalable AWS infrastructure, providing a practical framework for automating recruitment processes; and (3) it demonstrates the effectiveness of the system in enhancing accuracy, reducing bias, and promoting transparency in hiring.

The rest of this paper is organized as follows: Section 2 describes the proposed methodology and system architecture; Section 3 provides the experimental evaluation; Section 4 summarizes the contributions and future directions.

2. Methodology

This study presents a recruitment support system that integrates large language models (LLMs) with Retrieval Augmented Generation (RAG) to process job applications efficiently and accurately. The methodology is structured into two subsections: the general system architecture, which delineates the theoretical framework, and the implementation techniques, which describe the practical realization of data processing, query handling, and deployment. The system leverages a distributed computing environment to ensure scalability, security, and cost-efficiency. Below, we detail the architecture and implementation, supported by mathematical formulations, figures, and examples derived from the system design.

2.1. System Architecture

The system architecture is conceptualized as a modular framework that enhances LLM-generated responses through RAG, tailored for dynamic datasets such as job applications. The theoretical model comprises a vector database for semantic retrieval, an agent-based processing mechanism, and distributed storage and computation, as depicted in Figure 1. The architecture emphasizes modularity and scalability, abstracting components into interoperable modules. Each component is designed to address specific requirements of a recruitment support system, such as extracting relevant candidate information, matching skills to job criteria, and managing large volumes of data.

2.1.1. Retrieval-Augmented Generation (RAG)

RAG augments LLMs with non-parametric retrieval from external data sources, mitigating the limitations of fine-tuning for frequently updated datasets. In this system, RAG retrieves relevant job application data to ground LLM responses, reducing hallucinations and enhancing factual accuracy. The process involves embedding a user query q and retrieving a set of k relevant documents $D = d_1, d_2, \dots, d_k$ from a corpus C . Document relevance is determined using cosine similarity, as detailed in the Vector Database subsection. The retrieved documents are concatenated with the query to form the context $= [q; d_1; d_2; \dots; d_k]$, which is input to the LLM g_{LLM} to generate the response r :

$$r = g_{\text{LLM}}(\text{context}; \theta) \quad (1)$$

where θ denotes the LLM's parameters.

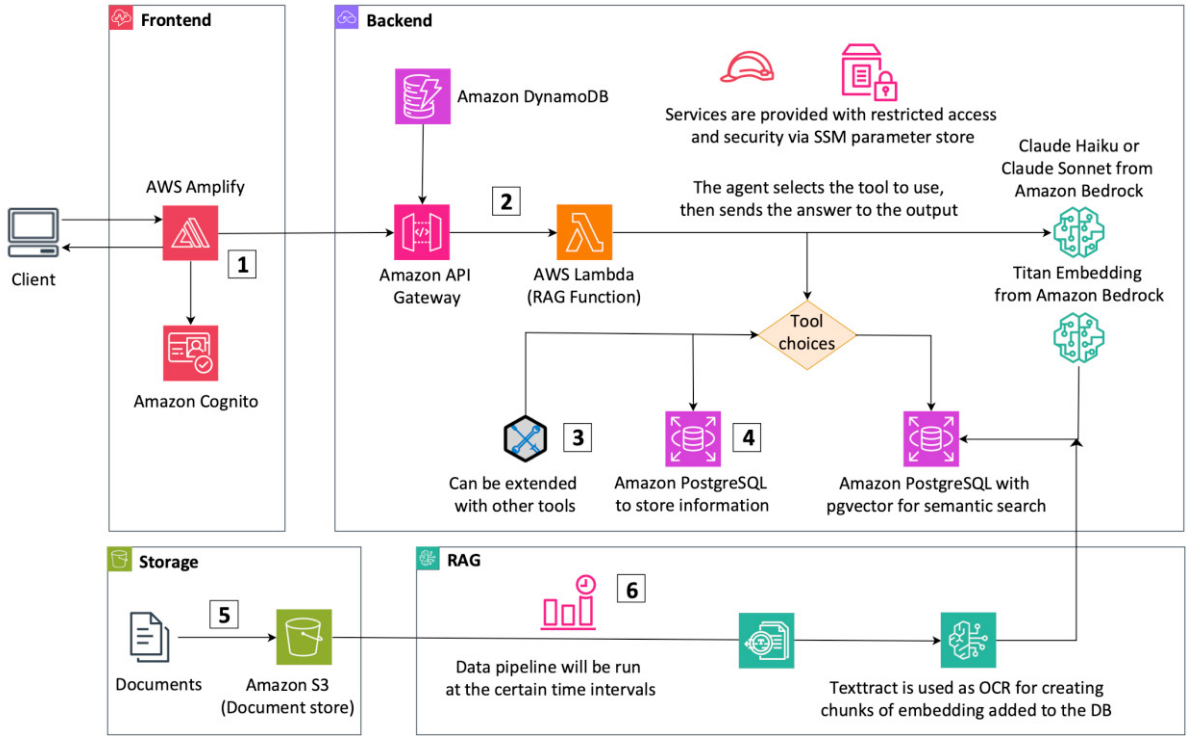


Fig. 1. General system architecture built on cloud computing infrastructure

RAG enables the system to retrieve specific candidate information (e.g., skills, experience) from unstructured CVs in response to recruiter queries (e.g., “Find candidates with AWS S3 experience”). This ensures responses are tailored to job requirements, facilitating accurate candidate screening and ranking.

2.1.2. Vector Database

The vector database stores embeddings of job application chunks, enabling efficient semantic retrieval for RAG queries. Each chunk, sized to balance context and efficiency, is embedded into a high-dimensional vector $e_i \in \mathbb{R}^n$ using an embedding model. Vectors are indexed using approximate nearest-neighbor (ANN) algorithms, such as Hierarchical Navigable Small World, achieving a retrieval time complexity of $O(\log |C|)$, where $|C|$ is the corpus size.

Semantic retrieval employs cosine to measure relevance between a query embedding e_q and a document embedding e_i :

$$\text{sim}(e_q, e_i) = \frac{\sum_{j=1}^n e_{q,j} \times e_{i,j}}{\sqrt{\sum_{j=1}^n e_{q,j}^2} \times \sqrt{\sum_{j=1}^n e_{i,j}^2}} \quad (2)$$

Where $e_{q,j}$ and $e_{i,j}$ are the j -th components of the query and document embeddings, and n is the embedding dimensionality. This yields a similarity score in $[-1, 1]$, with higher values indicating greater semantic alignment. The top- k chunks are retrieved, ensuring relevant content is prioritized.

Metadata (e.g., document ID, thematic labels) is associated with embeddings to filter the retrieval scope. For a query q , a metadata filter M (e.g., job category) reduces the search space:

$$C_M = d_i \hat{I} C | \text{metadata}(d_i) \hat{I} M \quad (3)$$

The vector database supports rapid retrieval of candidate data based on semantic queries, such as identifying CVs mentioning specific skills or roles. Metadata filters (e.g., “Software Engineering”) allow recruiters to narrow searches to relevant applicant pools, streamlining the identification of qualified candidates.

2.1.3. Agent-Based Processing

An agent, guided by the ReAct (Reasoning and Acting) framework, orchestrates query processing by selecting tools dynamically based on query requirements. The agent decomposes complex queries into subtasks, invoking RAG for retrieval, SQL for analysis, or external APIs for supplementary data. This process is modeled as a Markov Decision Process (MDP), with the state s_t representing the query context, action a_t denoting tool selection, and reward r_t reflecting output relevance. The agent optimizes the policy $\pi(a_t | s_t)$ to maximize cumulative reward, ensuring effective query resolution.

The agent enables flexible query handling, such as combining RAG-based CV retrieval with SQL-based statistical analysis (e.g., “Average years of experience for Python developers”). This supports complex recruitment tasks, like comparing candidate qualifications or generating shortlists based on multiple criteria.

2.1.4. Cloud-Based Storage and Computation

The architecture employs distributed storage and computation to support scalable, durable, and cost-efficient data management and processing. Storage is modeled as a distributed repository handling large volumes of unstructured data (e.g., job applications) with high durability, ensuring a low failure probability. Computation is event-driven, activating resources on demand to process queries or data tasks and optimizing resource utilization.

The distributed storage system accommodates large volumes of CVs and supporting documents, ensuring reliable access for retrieval and analysis. Event-driven computation supports real-time query processing (e.g., candidate searches) and batch processing of applications, enabling scalability for high-volume recruitment campaigns while maintaining cost efficiency.

2.2. System Implementation

The implementation operationalizes the theoretical model through data processing, query handling, and deployment, leveraging specific technologies to optimize RAG and LLM performance.

2.2.1. Data Processing

Data processing transforms raw job application files, primarily PDFs, into structured formats for RAG retrieval. The pipeline includes extraction, chunking, embedding, and metadata management, executed via a batch-processing pipeline to optimize resource usage. PDF files are processed using Unstructured and PyPDF libraries for structured text extraction. For complex layouts, Amazon Textract’s Optical Character Recognition (OCR) preserves structural integrity, as shown in Figure 2. Then, the text is segmented into chunks of 256 tokens, empirically determined to balance coherence and efficiency. Each chunk is embedded using the Amazon Titan Text Embeddings v2 model, generating semantic vectors stored in a PostgreSQL vector database. Metadata (e.g., document ID, file path, page number, job category) is attached. This chunk size minimizes LLM context window overload while ensuring semantic content for retrieval.

Metadata (e.g., years of experience, academic scores) is extracted using an LLM with Pydantic-based validation, enhanced by few-shot prompting. Job applications are stored in a PostgreSQL table.

2.2.2. Query Processing and Response Generation

Query processing employs RAG to deliver contextually relevant responses, as depicted in Figure 3. First, queries are formatted as prompts with a system message, conversation history, and tool descriptions, ensuring a professional tone. An example prompt is shown in Figure 4.

Applicant CV

Superhuman Fairness, UIC Department of Computer Science

June 2022 - Present

Research Assistant

- Applied post-processing from FairLearn framework on Adult dataset to mitigate unfairness in binary classification settings
- Reduced demographic disparity difference by 26.2% after applying post-processing, achieving a demographic disparity difference of 5.5%
- Spearheaded a data visualization project, and designed a plotting pipeline that increased efficiency by 25%.
- Accepted to ICML 2023 [\[Paper\]](#)[\[Code\]](#) – with Omid Memarrast and Brian Ziebart

OCR

Extracted entities using OCR

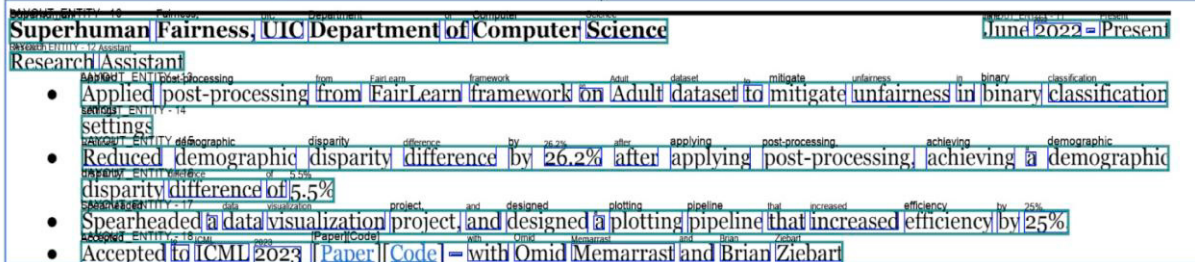


Fig. 2. General system architecture built on cloud computing infrastructure

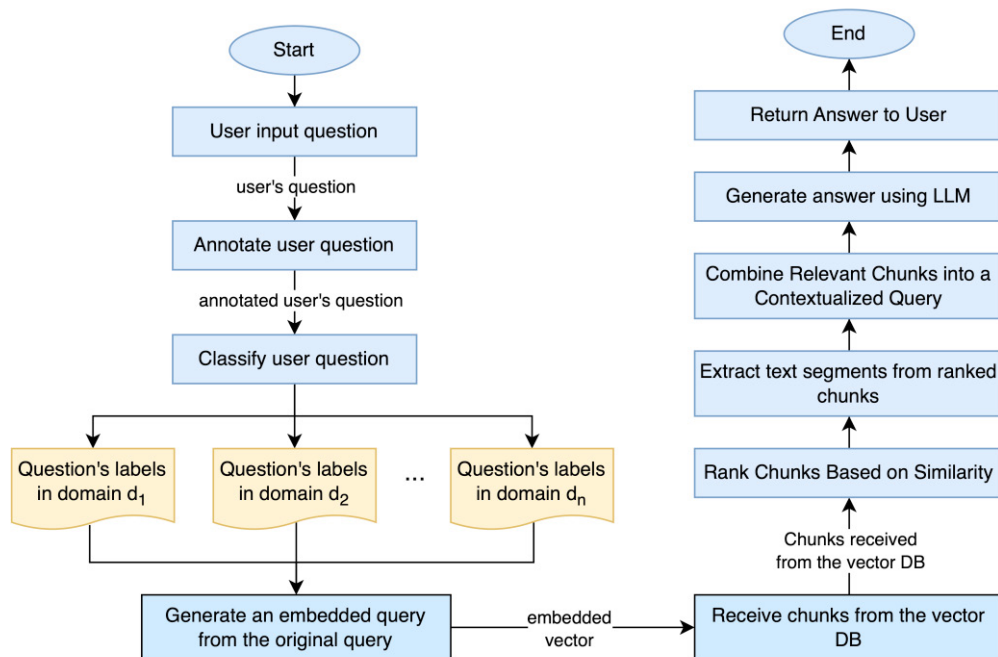


Fig. 3. RAG query processing workflow, showing embedding, retrieval, and response generation

Then, they are classified to select tools (e.g., RAG, SQL, web search). RAG queries use metadata tags (e.g., job category) to narrow the retrieval scope. Queries are embedded using the Titan Text Embeddings v2 model, and vectors are compared against the vector database using cosine similarity. The top- $k = 5$ chunks are retrieved. Retrieved chunks

are concatenated and input to the Anthropic Claude 3 Haiku model, selected for its balance of reasoning capability and cost-efficiency. Claude 3 Haiku is a lightweight model with a context window of up to 200,000 tokens, enabling it to process extensive RAG-generated contexts, such as concatenated CV chunks and query prompts. While the exact parameter count is proprietary, its design prioritizes efficiency for tasks like candidate data analysis. The model is configured with the following inference settings to minimize hallucination and ensure deterministic, accurate outputs: Temperature=0.99, Top-k=1, Top-p=1, Max Output Tokens=2048.

```
PROMPT: You are a friendly and knowledgeable AI assistant with a warm and approachable tone. Your goal is to provide helpful and accurate information to users while maintaining an engaging conversational style.
1. Use the conversation history in <conversation_history> to provide specific details and context, summarizing only the most relevant parts.
2. If information is insufficient, politely acknowledge the knowledge gap and suggest credible sources.
3. Adjust tone based on user communication style, maintaining professionalism.
4. Responses must be valid markdown within <markdown> tags.
```

Fig. 4. An example prompt

3. Experimental Results

To evaluate the performance of the proposed recruitment support system, we conducted a comprehensive experimental analysis using the Retrieval-Augmented Generation Assessment (RAGAS) framework [17]. This section presents the experimental setup, quantitative results, and a discussion of the system's user interface, demonstrating its efficacy in processing job applications and supporting recruitment tasks. The evaluation focuses on the integration of Retrieval-Augmented Generation (RAG) with the Anthropic Claude 3 Haiku model and Amazon Titan Text Embeddings v2, as described in the Methodology section.

3.1. Experimental Setup

The experimental dataset was meticulously curated by our research team to ensure relevance to real-world recruitment scenarios. We collected 800 curriculum vitae (CVs) from various recruitment campaigns, representing a diverse set of candidate profiles across industries such as software engineering, data science, and marketing. From these CVs, we generated a dataset of 400 test cases using the RAGAS framework, each designed to simulate typical recruiter queries about candidate qualifications, skills, and experiences. Each test case comprised:

- `user_input`: A query about a candidate's profile (e.g., "What programming and data management skills does An Pham possess?").
- `retrieved_contexts`: An array of (k=5) retrieved chunks, providing context for the response.
- `response`: The system's synthesized answer, leveraging RAG-augmented context.
- `reference`: Ground-truth information for validation. This field was either manually labeled by domain experts or inferred from a knowledge graph, ensuring accurate representation of candidate data.

The RAGAS framework evaluated system performance using four metrics, each formalized as follows [17]:

- **Context Recall**: Measures the proportion of relevant chunks retrieved:

$$\text{Context Recall} = \frac{|\text{relevant chunks} \cap \text{retrieved_contexts}|}{|\text{retrieved_contexts}|} \quad (4)$$

Where `relevant_chunks` are chunks in reference containing ground-truth information, and `retrieved_contexts` are the retrieved chunks. A score of 1.0 indicates all relevant chunks were retrieved.

- **Faithfulness**: Evaluates the proportion of claims in response inferable from retrieved contexts:

$$\text{Fairfulness} = \frac{|\text{claims}_{\text{response}} \cap \text{claims}_{\text{inferable}}|}{|\text{claims}_{\text{response}}|} \quad (5)$$

Where $\text{claims}_{\text{response}}$ is the set of factual claims in response, and $\text{claims}_{\text{inferable}}$ is the subset inferable from $\text{retrieved_contexts}$, assessed by Claude 3 Haiku.

- Factual Correctness: Approximates an F1 score of claims in response against reference:

$$\text{Factual Correctness} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

Where:

$$\circ \text{ Precision} = \frac{|\text{claims}_{\text{response}} \cap \text{claims}_{\text{reference}}|}{|\text{claims}_{\text{response}}|}, \text{ Recall} = \frac{|\text{claims}_{\text{response}} \cap \text{claims}_{\text{reference}}|}{|\text{claims}_{\text{reference}}|} \quad (7)$$

- $\text{claims}_{\text{reference}}$ is the set of ground-truth claims in reference.

- Semantic Similarity: Quantifies semantic alignment between response and reference:

$$\text{Semantic Similarity} = \cos(e_{\text{response}}, e_{\text{reference}}) = \frac{e_{\text{response}} \times e_{\text{reference}}}{|e_{\text{response}}| \times |e_{\text{reference}}|} \quad (8)$$

where e_{response} and $e_{\text{reference}}$ are embeddings from Amazon Titan Text.

These metrics were computed using Claude 3 Haiku, ensuring consistency with the response generation model. The dataset's diversity and manual curation enabled robust evaluation of the system's performance.

3.2. Quantitative Results

The system was tested on 400 test cases, with results summarized in Table 1 for a subset of representative queries. Average metric scores across all test cases were context recall = 0.9183, faithfulness = 0.882, factual correctness = 0.753, and semantic similarity = 0.827.

Table 1. Performance metrics for selected test cases, evaluated using the RAGAS framework.

User input	Retrieved context	Response	References	Context recall	Factual correctness	Faithfulness	Semantic similarity
What tools and techniques did An Pham use to c...	[An Pham utilized a combination of tools and p...	An Pham used Trifacta, MySQL, and Tableau to m...	Trifacta, MySQL, Tableau, campaign performance...	1.0	0.78	1.000000	0.817867
What academic achievements and recognitions di...	[While completing a Bachelor of Science in Bus...	During their undergraduate studies, An Pham gr...	Summa Cum Laude, Dean's List, advanced Honors ...	0.9	0.75	0.608736	0.665950
...	...	...	...	...	...	...	...

The high Context Recall (0.9183) indicates that the system retrieves relevant CV chunks in approximately 9 out of 10 cases, demonstrating effective semantic retrieval via the vector database and Amazon Titan Text Embeddings v2.

The Faithfulness score (0.882) suggests that generated responses closely adhere to retrieved contexts, with nearly 9 out of 10 claims inferable from the provided chunks. The Factual Correctness score (0.753) reflects moderate accuracy against ground-truth references, with about 7.5 out of 10 claims aligning with expected facts, potentially limited by the complexity of extracting nuanced claims from CVs. The Semantic Similarity score (0.827) indicates strong alignment between responses and references, though constrained by the dataset’s CV diversity and embedding model performance.

These results validate the system’s ability to respond accurately to recruitment queries, particularly for skill and experience retrieval, as evidenced by high recall and faithfulness. However, improvements in factual correctness and semantic similarity could be achieved by expanding the CV dataset or optimizing the embedding model, as discussed in the document.

3.3. System Interface and Practical Applicability

The recruitment support system’s user interface enhances its practical utility, enabling recruiters to interact with CV data efficiently. The key features include:

- **CV Query Chatbot:** Allows users to query CV data in Assistant Mode (LLM-only) or Agentic Mode (RAG or SQL-based). Users can clear conversation history to initial new contexts.
- **Recruitment Requirement Search:** Semantic vector matching identifies candidates matching job descriptions, with highlighted compatibility scores.
- **Keyword Search:** Full-text search in PostgreSQL enables precise keyword-based CV queries, with highlighted results.
- **Direct CV Interaction:** Users query specific CVs, generating interview questions or explanations of terms, leveraging extracted text as context.

Figure 5 illustrates the interfaces of two key functions: the CV Query Chatbot (screenshot on the left) and the Recruitment Requirement Search (screenshot on the right). Additional features include user authentication, storage of conversation history, management of API keys, job application uploads, job application management, and customizable Cloud Development Kit (CDK) components, all of which enhance the system’s flexibility. These interfaces translate the system’s high recall and accuracy into practical tools for recruiters, facilitating efficient candidate screening and streamlined data management.

4. Conclusion and Future Work

This paper has introduced a cloud-based applicant management system integrating Retrieval-Augmented Generation with large language models to efficiently handle and analyze large volumes of job applications. The system’s serverless AWS architecture ensures scalability and flexibility, while experimental results demonstrate strong performance in retrieving relevant information and generating accurate, context-aware responses. Experimental evaluations using the RAGAS framework demonstrated the system’s robust capability to retrieve relevant contextual information with an average context recall of 91.8%, while maintaining high levels of factual correctness (75.3%), faithfulness (88.2%), and semantic similarity (82.7%). These results confirm the effectiveness of integrating vector embeddings with LLMs to provide accurate and contextually grounded responses to user queries. The system’s modular design, including dynamic prompt engineering and tool integration via agents, further enhances adaptability and performance in real-world recruitment scenarios. Overall, the proposed approach effectively reduces recruiter workload and enhances the recruitment process, offering a practical solution for improving talent acquisition and business productivity.

Future research will focus on improving retrieval accuracy and efficiency through advanced re-ranking and summarization methods, as well as extending support for multilingual resumes to accommodate global recruitment needs.

Data Availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

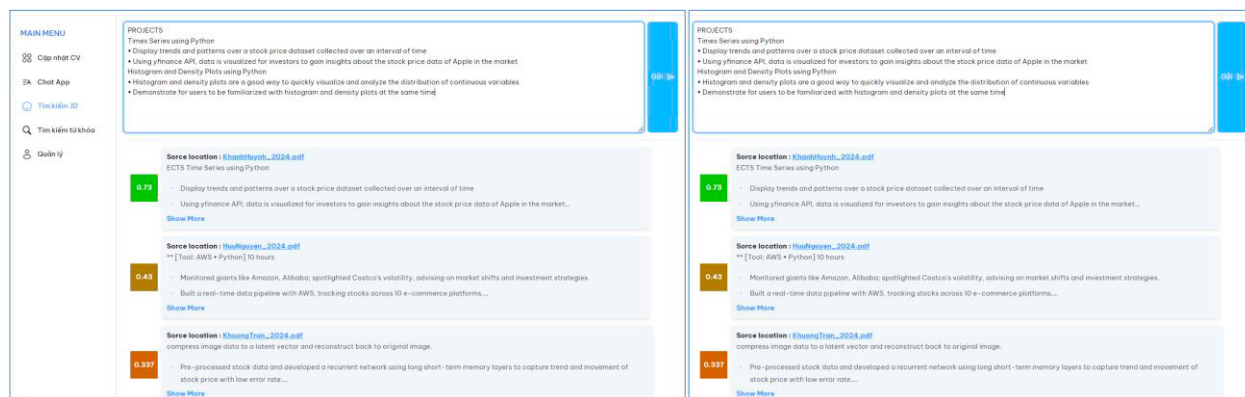


Fig. 5. Interface of the 'CV Query Chatbot' (left) and Recruitment Requirement Search (right) functions.

Author Contribution Statement

A.C. Tran: Conceptualization, Methodology, Data curation, Writing (original, review & editing), Software, Validation; C.N Tran: Methodology, Data curation, Software,

References

- [1] Javed F, Luo Q, McNair M, Jacob F, Zhao M, Kang TS. (2015) "Carotene: A job title classification system for the online recruitment domain." In: *2015 IEEE First International Conference on Big Data Computing Service and Applications*, IEEE, p. 286-93.
- [2] Devlin J, Chang MW, Lee K, Toutanova K. (2019) "Bert: Pre-training of deep bidirectional transformers for language understanding." In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies*, p. 4171-86.
- [3] Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, et al. (2020) "Language models are few-shot learners." *Advances in neural information processing systems*, 3: 1877-901.
- [4] Ponnaboyina R, Makala R, Venkateswara Reddy E. (2021) "Smart recruitment system using deep learning with natural language processing." In: *Intelligent Systems and Sustainable Computing: Proceedings of ICISSC 2021*. Springer, p. 647-55.
- [5] Devaraju S. (2022) "Natural Language Processing (NLP) in AI-Driven Recruitment Systems." *IJRSCEIT, DOI*, vol. 10.
- [6] Tian X, Pavur R, Han H, Zhang L. (2023) "A machine learning-based human resources recruitment system for business process management: using LSA, BERT and SVM." *Business Process Management Journal*, 29(1):202-22.
- [7] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. (2020) "Retrieval-augmented generation for knowledge-intensive NLP tasks." *Advances in neural information processing systems*, 33:9459-74.
- [8] Mikolov T, Chen K, Corrado G, Dean J. (2013) "Efficient estimation of word representations in vector space." *arXiv preprint arXiv:13013781*.
- [9] Pennington J, Socher R, Manning CD. (2014) "Glove: Global vectors for word representation." In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, p. 1532-43.
- [10] Reimers N, Gurevych I. (2019) "Sentence-bert: Sentence embeddings using siamese bert-networks." *arXiv preprint arXiv:190810084*.
- [11] Qin C, Zhu H, Xu T, Zhu C, Jiang L, Chen E, et al. (2018) "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach." In: *The 41st international ACM SIGIR conference on research & development in information retrieval*, p. 25-34.
- [12] Wang Y, Ma X, Chen W. (2023) "Augmenting Black-box LLMs with Medical Textbooks for Biomedical Question Answering." *arXiv preprint arXiv:230902233*.
- [13] Ghanbari Haez S, Segala M, Bellan P, Magnolini S, Sanna L, Consolandi M, et al. (2024) "A retrieval-augmented generation strategy to enhance medical chatbot reliability." In: *International Conference on Artificial Intelligence in Medicine*, Springer, p. 213-23.
- [14] Chaudhuri S. (1998) "An overview of query optimization in relational systems." In: *Proceedings of the seventeenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, p. 34-43.
- [15] Buyya R, Srirama SN, Casale G, Calheiros R, Simmhan Y, Varghese B, et al. (2018) "A manifesto for future generation cloud computing: Research directions for the next decade." *ACM computing surveys (CSUR)*, 51(5):1-38.
- [16] Xiong S, Chen X, Zhang H. (2023) "Deep learning-based multifunctional end-to-end model for optical character classification and denoising." *Journal of Computational Methods in Engineering Applications*, 2023:1-13.
- [17] Es S, James J, Anke LE, Schockaert S. (2024) "Ragas: Automated evaluation of retrieval augmented generation." In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, p. 150-8.